UNIT II

Mathematical Computing with Python (NumPy):Working with NumPy Arrays, Data Types, Array Creation, Indexing and Slicing, Numerical Operations on Arrays, Array Functions, Data Processing using Arrays**,** Loading and Saving Data, Saving an Array, Loading an Array, Numpy Random Numbers

Data Manipulation with Pandas: Data Wrangling, Data Exploration, Cleaning Data, Filtering, Merging Data, Reshaping Data, Data Aggregation**,** Reading and Writing Files, Loading and Saving Data with Pandas

NUMPY

NumPy is a Python library.

NumPy is used for working with arrays.

NumPy is short for "Numerical Python"

NumPy(Numerical Python) is a fundamental library for Python **numerical computing**. It provides efficient multi-dimensional array objects and various mathematical functions for handling large datasets making it a critical tool for professionals in fields that require heavy computation.

Creating NumPy Arrays

- **Using ndarray :** The array object is called ndarray. NumPy arrays are created using the array() function.

```
import numpy as np

# Creating a 1D array
x = np.array([1, 2, 3])

# Creating a 2D array
y = np.array([[1, 2], [3, 4]])

# Creating a 3D array
z = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

```
print(x)
print(y)
print(z)
```

```
[1 2 3]
[[1 2]
 [3 4]]
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

NumPy Array Indexing

Knowing the basics of NumPy array indexing is important for analyzing and manipulating the array object.

- **Basic Indexing:** Basic indexing in NumPy allows you to access elements of an array using indices.

Example:

```
1
import numpy as np
2


3
# Create a 1D array
4
arr1d = np.array([10, 20, 30, 40, 50])
5


6
```

```
# Single element access
7
print("Single element access:", arr1d[2])
8


9
# Negative indexing
10
print("Negative indexing:", arr1d[-1])
11


12
# Create a 2D array
13
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
14


15
# Multidimensional array access
16
print("Multidimensional array access:", arr2d[1, 0])
```

Output

Single element access: 30

Negative indexing: 50

Multidimensional array access: 4

- **Slicing:** Just like lists in Python, NumPy arrays can be sliced. As arrays can be multidimensional, you need to specify a slice for each dimension of the array.

Example:

```
1
```

```
import numpy as np
2


3
arr = np.array([[1, 2, 3], [4, 5, 6]])
4
#elements from index 1 to 3
5
print("Range of Elements:",arr[1:4])
6


7
#all rows, second column
8
print("Multidimensional Slicing:", arr[:, 1])
```

Output

Range of Elements: [[4 5 6]]

Multidimensional Slicing: [2 5]

- **Advanced Indexing:** Advanced Indexing in NumPy provides more powerful and flexible ways to access and manipulate array elements.

Example:

```
1
import numpy as np
2
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
3


4
# Integer array indexing
5
```

```
indices = np.array([1, 3, 5])
6
print ("Integer array indexing:", arr[indices])
7


8
# boolean array indexing
9
cond = arr > 0
10
print ("\nElements greater than 0:\n", arr[cond])
```

Output

```
Elements at indices (0, 3), (1, 2), (2, 1),(3, 0):
 [4. 6. 0. 3.]


Elements greater than 0:
 [2.  4.  4.  6.  2.6 7.  8.  3.  4.  2. ]
```

NumPy Basic Operations

Element-wise operations in NumPy allow you to perform mathematical operations on each element of an array individually, without the need for explicit loops.

- **Element-wise Operations:** We can perform arithmetic operations like addition, subtraction, multiplication, and division directly on NumPy arrays.

Example:

```
1
import numpy as np
2


3
x = np.array([1, 2, 3])
4
```

```
y = np.array([4, 5, 6])
5

6
# Addition
7
add = x + y
8
print("Addition:",add)
9

10
# Subtraction
11
subtract = x - y
12
print("substration:",subtract)
13

14
# Multiplication
15
multiply = x * y
16
print("multiplication:",multiply)
17

18
# Division
19
```

```
divide = x / y
20
print("division:", divide)
```

Output

```
Addition: [5 7 9]
substration: [-3 -3 -3]
multiplication: [ 4 10 18]
division: [0.25 0.4  0.5 ]
```

- **Unary Operation:** These operations are applied to each individual element in the array, without the need for multiple arrays (as in binary operations).

Example:

```
1
import numpy as np
2


3
# Example array with both positive and negative values
4
arr = np.array([-3, -1, 0, 1, 3])
5


6
# Applying a unary operation: absolute value
7
result = np.absolute(arr)
8
print("Absolute value:", result)
```

Output

```
Absolute value: [3 1 0 1 3]
```

- **Binary Operators:** Numpy Binary Operations apply to the array elementwise and a new array is created. We can use all basic arithmetic operators like +, -, /, etc. In the case of +=, -=, = operators, the existing array is modified.

Example:

```
1
import numpy as np
2


3
# Two example arrays
4
arr1 = np.array([1, 2, 3])
5
arr2 = np.array([4, 5, 6])
6


7
# Applying a binary operation: addition
8
result = np.add(arr1, arr2)
9


10
print("Array 1:", arr1)
11
print("Array 2:", arr2)
12
print("Addition Result:", result)
```

Output

```
Array 1: [1 2 3]
```

Array 2: [4 5 6]

Addition Result: [5 7 9]

NumPy ufuncs

NumPy provides familiar mathematical functions such as **sin, cos, exp, etc**. These functions also operate elementwise on an array, producing an array as output.

Example:

import numpy as np

create an array of sine values

a = np.array([0, np.pi/2, np.pi])

print ("Sine values of array elements:", np.sin(a))

exponential values

a = np.array([0, 1, 2, 3])

print ("Exponent of array elements:", np.exp(a))

square root of array values

print ("Square root of array elements:", np.sqrt(a))

Output:

Sine values of array elements: [0.00000000e+00 1.00000000e+00 1.22464680e-16]

Exponent of array elements: [1. 2.71828183 7.3890561 20.08553692]

Square root of array elements: [0. 1. 1.41421356 1.73205081]

NumPy Sorting Arrays

We can use a simple **np.sort()** method for sorting Python NumPy arrays.

Example:

1

import numpy as np

2

3

```
# set alias names for dtypes
4
dtypes = [('name', 'S10'), ('grad_year', int), ('cgpa', float)]
5


6
# Values to be put in array
7
values = [('Hrithik', 2009, 8.5), ('Ajay', 2008, 8.7),
8
          ('Pankaj', 2008, 7.9), ('Aakash', 2009, 9.0)]
9


10
# Creating array
11
arr = np.array(values, dtype = dtypes)
12
print ("\nArray sorted by names:\n",
13
           np.sort(arr, order = 'name'))
14


15
print ("Array sorted by graduation year and then cgpa:\n",
16
               np.sort(arr, order = ['grad_year', 'cgpa']))
```

Output

```
Array sorted by names:
 [(b'Aakash', 2009, 9. ) (b'Ajay', 2008, 8.7) (b'Hrithik', 2009, 8.5)
```

(b'Pankaj', 2008, 7.9)]

Array sorted by graduation year and then cgpa:

[(b'Pankaj', 2008, 7.9) (b'Ajay',...

Key Features of NumPy

NumPy has various features that make it popular over lists.

- **N-Dimensional Arrays**: NumPy's core feature is ndarray, a powerful N-dimensional array object that supports homogeneous data types.
- **Arrays with High Performance**: Arrays are stored in contiguous memory locations, enabling faster computations than Python lists (Please see Numpy Array vs Python List for details).
- Broadcasting: This allows element-wise computations between arrays of different shapes. It simplifies operations on arrays of **various shapes** by automatically aligning their dimensions without creating new data.
- Vectorization: Eliminates the need for explicit Python loops by applying operations directly on entire arrays.
- **Linear algebra**: NumPy contains routines for linear algebra operations, such as matrix multiplication, decompositions, and determinants.

Basic

Introduction

Getting Started

Creating Arrays

Array Indexing

Array Slicing

Data Types

Copy vs View

Array Shape

Array Reshape

Array Iterating

Array Join

Array Split

Array Search

Array Sort

Array Filter

Random

Random Intro

Data Distribution

Random Permutation

Seaborn Module

Normal Dist.

Binomial Dist.

Poisson Dist.

Uniform Dist.

Logistic Dist.

Multinomial Dist.

Exponential Dis.

Chi Square Dist.

Rayleigh Dist.

Pareto Dist.

Zipf Dist.

ufunc

ufunc Intro

Create Function

Simple Arithmetic

Rounding Decimals

Logs

Summations

Products

Differences

Finding LCM

Finding GCD

Trigonometric

Hyperbolic

Set Operations

NumPy is a powerful Python library that can manage different types of data. Here we will explore the **Datatypes in NumPy** and How we can check and create datatypes of the NumPy array.

DataTypes in NumPy

A **data type** in NumPy is used to specify the type of data stored in a variable. Here is the list of characters available in NumPy to represent data types.

Character	Meaning
b	Boolean
f	Float
m	Time Delta
O	Object
U	Unicode String
i	Integer
u	Unsigned Integer
c	Complex Float
M	DateTime
S	String
V	A fixed chunk of memory for other types (void)

The list of various types of data types provided by NumPy are given below:

Data Type	Description
bool_	Boolean
int_	Default integer type (int64 or int32)
intc	Identical to the integer in C (int32 or int64)
intp	Integer value used for indexing
int8	8-bit integer value (-128 to 127)
int16	16-bit integer value (-32768 to 32767)
int32	32-bit integer value (-2147483648 to 2147483647)
int64	64-bit integer value (-9223372036854775808 to 9223372036854775807)
uint8	Unsigned 8-bit integer value (0 to 255)
uint16	Unsigned 16-bit integer value (0 to 65535)
uint32	Unsigned 32-bit integer value (0 to 4294967295)
uint64	Unsigned 64-bit integer value (0 to 18446744073709551615)
float_	Float values
float16	Half precision float values

Data Type	Description
float32	Single-precision float values
float64	Double-precision float values
complex_	Complex values
complex64	Represent two 32-bit float complex values (real and imaginary)
complex128	Represent two 64-bit float complex values (real and imaginary)

Checking the Data Type of NumPy Array

We can check the datatype of Numpy array by using dtype. Then it returns the data type all the elements in the array.

```
1
import numpy as np
2


3
# Create a NumPy array
4
arr = np.array([1, 2, 3, 4, 5])
5


6
# Check the data type of the array
7
data_type = arr.dtype
8
print(data_type)
```

Output

int64

Create Arrays With a Defined Data Type

We can create an array with a defined data type by specifying “dtype” attribute in **numpy.array()** method while initializing an array.

```
1
import numpy as np
2


3
arr1 = np.array([1, 2, 3, 4], dtype=np.float64)
4


5
# Creating a 3x3 int32 array of zeros
6
arr2 = np.zeros((3, 3), dtype=np.int32)
7


8
# Creating a 2x2 complex128 array of ones
9
arr3 = np.ones((2, 2), dtype=np.complex128)
10


11
# Creating a 1D bool array
12
arr4 = np.empty((4,), dtype=np.bool_)
13
```

```
14
# Print the arrays and their data types
15
print(arr1.dtype)
16
print(arr2.dtype)
17
print(arr3.dtype)
18
print(arr4.dtype)
```

Output

```
float64
int32
complex128
bool
```

Convert Data Type of NumPy Arrays

We can convert data type of an arrays from one type to another using astype() function.

```
1
import numpy as np
2

3
arr1 = np.array([1.2, 2.5, 3.7])
4

5
# Converting to int32
6
```

```
arr2 = arr1.astype(np.int32)
7


8
# Print new array and its type
9
print(arr2)
10
print(arr2.dtype)
```

Output

```
[1 2 3]
int32
```

NumPy Operations

Python package NumPy stands for "Numerical Python." The logical computation package includes an efficient N-D array object and provides facilities to link C, C++, and other programming languages. Additionally, it helps with arbitrary number capacity, linear-based math, and other subjects. NumPy displays can be used as a powerful multi-dimensional container for general data. Python Array: Rows and columns make up the robust N-dimensional object known as a Numpy array. We may access its elements and initialize NumPy arrays using a nested Python list. On a structural plane, a Numpy array consists of a mixture of:

- The Data reference displays the memory location of the Numpy array's initial byte.
- The name Data type or the dtype argument describes the Python types of elements stored within the particular array.
- The shape denotes the number of rows and columns the array has.
- The strides specify how many bytes in the system's memory should be bypassed before moving on to the next array element.

Numpy Basics

Array of Ones

Creates a NumPy array according to the parameters given, with all elements being 1.

Code

```
1. # Python program to create a numpy array of ones
2. array = np.ones([2,3])
3. print(array)
```

Output

Backward Skip 10sPlay VideoForward Skip 10s

[[1. 1. 1.]

[1. 1. 1.]]

Array of Zeros

This function returns a NumPy array with all entries set to 0 having dimensions as specified.

Code

```
1. # Python program to create a numpy array of zeroes
2. array = np.zeros([2,3])
3. print(array)
```

Output

[[0. 0. 0.]

[0. 0. 0.]]

These functions are simple, and they can be used to create sample arrays which are often needed for various computational purposes.

Eye

Let's now examine how the eye method works. A 2-D array containing ones on its diagonal and zeros everywhere else is this function's output.

Code

Advertisement

```
1. # Python program to create a numpy array having ones on diagonal and zero
   es elsewhere
2. array = np.eye(4)
3. print(array)# Python program to create a numpy array of ones
4. array = np.ones([2,3])
5. print(array)
```

Output

[[1. 0. 0. 0.]

[0. 1. 0. 0.]

[0. 0. 1. 0.]

[0. 0. 0. 1.]]

Similarly, the diag() method builds a 2D array with all other members set to zero, and the items supplied to function as arguments are set to the diagonal elements.

Code

```
1. import numpy as np
2. y = np.array([78,56,89])
3. np.diag(y)
```

Output

array([[78, 0, 0],

[0, 56, 0],

[0, 0, 89]])

Vstack() and Hstack()

We can use the vstack() function to vertically stack two arrays and hstack() to stack two or more arrays horizontally. Let's practice with several instances.

Code

```
# Python program to stack two arrays

array1 = np.array([[2,3,4,5],[4,3,5,3]])
array2 = np.array([[6,3,5,2],[8,2,5,8]])

# Vertical stacking
array = np.vstack((array1, array2))
print("Vertically stacked: \n", array)

# Horizontal stacking
array = np.hstack((array1, array2))
print("Horizontally stacked: \n", array)
```

Output

Advertisement

Vertically stacked:

[[2 3 4 5]

[4 3 5 3]

[6 3 5 2]

[8 2 5 8]]

Horizontally stacked:

[[2 3 4 5 6 3 5 2]

[4 3 5 3 8 2 5 8]]

Operations on Numpy Array

Arithmetic Operations

Code

```
# Python program to perform arithmetic operations on the Numpy arrays

import numpy as np
```

```
4.
5.  # Initializing our array
6.  array1 = np.arange(9, dtype = np.float_).reshape(3, 3)
7.  print('First Array:')
8.  print(array1)
9.
10. print('Second array:')
11. array2 = np.arange(11,20, dtype = np.float_).reshape(3, 3)
12. print(array2)
13.
14. print('\nAdding two arrays:')
15. print(np.add(array1, array2))
16.
17. print('\nSubtracting two arrays:')
18. print(np.subtract(array1, array2))
19.
20. print('\nMultiplying two arrays:')
21. print(np.multiply(array1, array2))
22.
23. print('\nDividing two arrays:')
24. print(np.divide(array1, array2))
```

Output

First Array:

[[0. 1. 2.]

[3. 4. 5.]

[6. 7. 8.]]

Second array:

[[11. 12. 13.]

[14. 15. 16.]

[17. 18. 19.]]

Adding two arrays:

[[11. 13. 15.]

[17. 19. 21.]

[23. 25. 27.]]

Subtracting two arrays:

[[-11. -11. -11.]

[-11. -11. -11.]

[-11. -11. -11.]]

Multiplying two arrays:

[[0. 12. 26.]

[42. 60. 80.]

[102. 126. 152.]]

Dividing two arrays:

[[0. 0.08333333 0.15384615]

[0.21428571 0.26666667 0.3125]

[0.35294118 0.38888889 0.42105263]]

numpy.reciprocal()

This method returns the argument's element-by-element inverse. When an element's absolute value is greater than 1, the outcome is always 0, and an overflow warning is shown for integer 0.

Advertisement

Code

```
1. # Python program to show how to perform the reciprocal operation on the Nu
   mPy arrays
2.
3. # Importing the numpy library
4. import numpy as np
```

```
5.
6.  # Initializing our array
7.  array = np.array([23, 14, 63.9, 23.5, 23.7, 13, 7])
8.
9.  print('The array is:')
10. print(array)
11.
12. print('\nAfter applying the reciprocal function array is:')
13. print(np.reciprocal(array))
14.
15. # Creating another array
16. array2 = np.array([24], dtype = int)
17.
18. print()
19.
20. print('The second array is: ')
21. print(array2)
22.
23. print('After applying the reciprocal function the array is: ')
24. print(np.reciprocal(array2))
```

Output

The array is:

[23. 14. 63.9 23.5 23.7 13. 7.]

After applying the reciprocal function array is:

[0.04347826 0.07142857 0.01564945 0.04255319 0.04219409 0.07692308

0.14285714]

The second array is:

[24]

After applying the reciprocal function the array is:

[0]

numpy.power()

This function treats the original array's elements as the base in the exponents' syntax, which then raises them to the power of the adjacent elements provided in the second array argument.

Code

```
# Python program to show how to take powers of the numpy array

# Importing the numpy library
import numpy as np

# Initializing the array
array = np.array([3, 5, 2])

print('The original array is:')
print(array)

print('Applying the numpy power function: ')
print(np.power(array, 2))

# Initializing another array
print('The second array is:')
array_ = np.array([2, 4, 5])
print(array_)

# Giving the power as a numpy array
print('Applying the numpy power function: ')
print(np.power(array, array_))
```

Output

The original array is:

[3 5 2]

Applying the numpy power function:

[9 25 4]

The second array is:

[2 4 5]

Applying the numpy power function:

[9 625 32]

numpy.mod()

This function returns the remainder of the division of the corresponding elements in the input array. The function numpy.remainder() also produces the same result.

Code

```
# Python program to show how to find remainders on dividing an array by an array in numpy

# Importing the numpy library
import numpy as np

# Initializing the arrays
array = np.array([5, 10, 15])
array1 = np.array([2, 4, 5])

print('The original array:')
print(array)

print('The array for diving the original array:')
print(array1)

print('Applying the numpy mod function:')
print(np.mod(array, array1))

print('Applying the numpy remainder function:')
print(np.remainder(array, array1))
```

Output

The original array:

[5 10 15]

The array for diving the original array:

[2 4 5]

Applying the numpy mod function:

[1 2 0]

Applying the numpy remainder function:

[1 2 0]

The following functions are used to perform operations on an array with complex numbers.

numpy.real() ? This function will return the real part of the given complex argument.

numpy.imag() ? This function will return the imaginary part of the complex argument.

numpy.conj() ? This function will return the complex conjugate of the given complex argument. It is obtained by swapping the sign of the imaginary part.

numpy.angle() ? This function will return the angle of the given complex argument. The function has a parameter having the keyword- degree. If set to true, the function will return the angle in degrees; otherwise, the angle is returned in radians.

Code

```
# Python program to show how to perform operations on the complex element
s of an array in numpy

# Importing the numpy library
import numpy as np

# Initializing the array
a = np.array([-6.6j, 0.9j, 14. , 1+9j])

print('Our complex array is:')
print(a)

```

```
12. print('Applying the numpy real function: ')
13. print(np.real(a))
14.
15. print('Applying the numpy imag function: ')
16. print(np.imag(a))
17.
18. print('Applying the numpy conj function: ')
19. print(np.conj(a))
20.
21. print('Applying the numpy angle function: ')
22. print(np.angle(a))
23.
24. print('Applying the numpy angle function again (result in degrees)')
25. print(np.angle(a, deg = True))
```

Output

Our complex array is:

[-0.-6.6j 0.+0.9j 14.+0.j 1.+9.j]

Applying the numpy real function:

[-0. 0. 14. 1.]

Applying the numpy imag function:

[-6.6 0.9 0. 9.]

Applying the numpy conj function:

[-0.+6.6j 0.-0.9j 14.-0.j 1.-9.j]

Applying the numpy angle function:

[-1.57079633 1.57079633 0. 1.46013911]

Applying the numpy angle function again (result in degrees)

[-90. 90. 0. 83.65980825]

Using Numpy Arrays with Conditional Expressions

To identify the values that meet your criterion, utilize conditionals. The outcome of a conditional operating condition is also a numpy array because "array" is also a

numpy array. Our conditional check produces an array containing boolean values as its result.

Code

```
# Python program to perform condition operations on the array

# Initializing the array
array = np.array([20, 30, 23, 12, 53, 54, 24])

# Values greater than 30
array_30 = array >= 30
print(array_30)

# Only storing the values that are greater than 30
print(array[array_30])

# Direct method to do the above task
print(array[array >= 30])
```

Output

[False True False False True True False]

[30 53 54]

[30 53 54]

numpy.dot()

We will begin with the cases in which both arguments are scalars or one-dimensional arrays.

Code

```
# Python program to find the dot product of the numpy vectors

# Directly passing the values
print(np.dot(3, 4))

# Initializing the two arrays of a single value
```

```
a = np.array([3])
b = np.array([4])
print("Dimensions of a: ", a.ndim)
print(f"The dot product of {a} and {b} is: ", np.dot(a, b))

# Initializing the two arrays of two value
a = np.array([4, -3])
b = np.array([-6, 3])
print(f"The dot product of {a} and {b} is: ", np.dot(a, b))
```

Output

12

Dimensions of a: 1

The dot product of [3] and [4] is: 12

The dot product of [4 -3] and [-6 3] is: -33

Logical Operators

The logical operators "or" and "and" also apply to numpy arrays elementwise. For this, we can use the numpy logical_or and logical_and methods.

Code

```
# Python program to use logical operations on numpy arrays

# Importing the numpy library
import numpy as np

# Intializing the arrays
array = np.array([ [True, False], [True, False]])
array1 = np.array([ [False, True], [True, False]])

# Applying numpy "or" and "and" operations
print("After or operation: \n", np.logical_or(array, array1))
print("After and operation: \n", np.logical_and(array, array1))
```

Output

After or operation:

[[True True]

[True False]]

After and operation:

[[False False]

[True False]]

Implementing Operations on Arrays Having Different Shapes

Up until now, we have dealt with two distinct instances using simple operations such as "+" or "*":

- an operation performed on a scalar and a numpy array
- an operation used to combine two arrays having the same shape.

The next section will demonstrate that we can still apply operators even if an array has a unique shape. But only particular circumstances allow it to function.

Broadcasting

Arrays of different dimensions can be used for arithmetic operations thanks to Numpy's robust Broadcasting technique. This implies that we take a larger dimension array and a smaller dimension array, and we convert or extend the smaller dimension array to the larger dimension array multiple times to carry out an operation. To put this in another way, the smaller array can occasionally be "broadcasted" so that it takes on the same dimension as the larger array.

In our Python software, we can avoid loops with the help of broadcasting. In the C-based Numpy implementations, the iteration happens implicitly. Additionally, we refrain from making duplicates of our data.

A straightforward example is used to illustrate the broadcasting operational principle.

Code

```
# Python program to show how to apply operations on two numpy arrays of dif
ferent shapes

# Importing the numpy library
import numpy as np

# Initializing two arrays of different lengths
arr1 = np.array([ [3, 2, 4], [1, 3, 2], [5, 3, 3] ])
arr2 = np.array([2, 2, 2])
```

```
9.
10. # Original arrays
11. print("arr1: \n", arr1)
12. print("arr2: \n", arr2)
13.
14. # Multiplieng the arrays
15. print("Multiplication using broadcasting: ")
16. print(arr1 * arr2)
17.
18. # Adding the arrays
19. print("Addition using broadcasting: ")
20. print(arr1 + arr2)
```

Output

arr1:

[[3 2 4]

[1 3 2]

[5 3 3]]

arr2:

[2 2 2]

Multiplication using broadcasting:

[[6 4 8]

[2 6 4]

[10 6 6]]

Addition using broadcasting:

[[5 4 6]

[3 5 4]

[7 5 5]]

Distance Matrix

A distance matrix in geometry is a matrix or a two-dimensional array that stores the distances between the members of a set, pairwise taken, in mathematics, computer

science, and particularly in graph theory. If the set has n elements, this two-dimensional array's size is n x n.

A distance matrix connecting the coordinates, in our case, the alphabet, to a hypothetical point:

Code

```
1.  # Python program to find the distance matrix using numpy functions
2.
3.  # Importing numpy
4.  import numpy as np
5.  # Initializing the arrays
6.  coord = ["A", "B", "C", "D", "E",
7.           "F", "G", "H", "I", "J",
8.           "K", "L", "M", "N", "O",
9.           "P", "Q", "R", "S", "T",
10.          "U", "V", "W", "X", "Y", "Z"]
11.
12. distance_from_coord = [12, 14, 13, 18, 18,
13.                        16, 69, 12, 23, 24,
14.                        10, 45, 35, 27, 15,
15.                        33, 34, 37, 28, 22,
16.                        47, 18, 19, 17, 21, 14]
17.
18. distances =  np.array(distance_from_coord[:12])
19. print(distances)
20. print(np.abs(distances - distances[:, np.newaxis]))
```

Output

[12 14 13 18 18 16 69 12 23 24 10 45]

[[0 2 1 6 6 4 57 0 11 12 2 33]

[2 0 1 4 4 2 55 2 9 10 4 31]

[1 1 0 5 5 3 56 1 10 11 3 32]

[6 4 5 0 0 2 51 6 5 6 8 27]

[6 4 5 0 0 2 51 6 5 6 8 27]

[4 2 3 2 2 0 53 4 7 8 6 29]

[57 55 56 51 51 53 0 57 46 45 59 24]

[0 2 1 6 6 4 57 0 11 12 2 33]

[11 9 10 5 5 7 46 11 0 1 13 22]

[12 10 11 6 6 8 45 12 1 0 14 21]

[2 4 3 8 8 6 59 2 13 14 0 35]

[33 31 32 27 27 29 24 33 22 21 35 0]]

Numerical Operations on Numpy Arrays

NumPy is a Python package which means 'Numerical Python'. It is the library for logical computing, which contains a powerful n-dimensional array object, gives tools to integrate C, C++ and so on. It is likewise helpful in linear based math, arbitrary number capacity and so on. NumPy exhibits can likewise be utilized as an effective multi-dimensional compartment for generic data. **NumPy Array:** Numpy array is a powerful N-dimensional array object which is in the form of rows and columns. We can initialize **NumPy arrays** from nested Python lists and access it elements. A Numpy array on a structural level is made up of a combination of:

- The **Data** pointer indicates the memory address of the first byte in the array.
- The **Data type** or **dtype** pointer describes the kind of elements that are contained within the array.
- The **shape** indicates the shape of the array.
- The **strides** are the number of bytes that should be skipped in memory to go to the next element.

Operations on Numpy Array

Arithmetic Operations:

- Python3

```
# Python code to perform arithmetic
# operations on NumPy array



import numpy as np



# Initializing the array
arr1 = np.arange(4, dtype = np.float_).reshape(2, 2)


print('First array:')
print(arr1)
```

```
print('\nSecond array:')
arr2 = np.array([12, 12])
print(arr2)

print('\nAdding the two arrays:')
print(np.add(arr1, arr2))

print('\nSubtracting the two arrays:')
print(np.subtract(arr1, arr2))

print('\nMultiplying the two arrays:')
print(np.multiply(arr1, arr2))

print('\nDividing the two arrays:')
print(np.divide(arr1, arr2))
```

Output:

```
First array:
[[ 0.  1.]
 [ 2.  3.]]

Second array:
[12 12]

Adding the two arrays:
[[ 12.  13.]
 [ 14.  15.]]

Subtracting the two arrays:
[[-12. -11.]
```

```
 [-10.  -9.]]
```

Multiplying the two arrays:

```
[[  0.  12.]
 [ 24.  36.]]
```

Dividing the two arrays:

```
[[ 0.          0.08333333]
 [ 0.16666667  0.25      ]]
```

numpy.reciprocal() This function returns the reciprocal of argument, element-wise. For elements with absolute values larger than 1, the result is always 0 and for integer 0, overflow warning is issued. **Example:**

- Python3

```
# Python code to perform reciprocal operation
# on NumPy array
import numpy as np
arr = np.array([25, 1.33, 1, 1, 100])

print('Our array is:')
print(arr)

print('\nAfter applying reciprocal function:')
print(np.reciprocal(arr))

arr2 = np.array([25], dtype = int)
print('\nThe second array is:')
print(arr2)

print('\nAfter applying reciprocal function:')
print(np.reciprocal(arr2))
```

Output

Our array is:

[25. 1.33 1. 1. 100.]

After applying reciprocal function:

[0.04 0.7518797 1. 1. 0.01]

The second array is:

[25]

After applying reciprocal function:

[0]

numpy.power() This function treats elements in the first input array as the base and returns it raised to the power of the corresponding element in the second input array.

- Python3

```
# Python code to perform power operation
# on NumPy array


import numpy as np


arr = np.array([5, 10, 15])

print('First array is:')
print(arr)

print('\nApplying power function:')
print(np.power(arr, 2))
```

```
print('\nSecond array is:')
arr1 = np.array([1, 2, 3])
print(arr1)

print('\nApplying power function again:')
print(np.power(arr, arr1))
```

Output:

First array is:

[5 10 15]

Applying power function:

[25 100 225]

Second array is:

[1 2 3]

Applying power function again:

[5 100 3375]

numpy.mod() This function returns the remainder of division of the corresponding elements in the input array. The function numpy.remainder() also produces the same result.

- Python3

```
# Python code to perform mod function
# on NumPy array

import numpy as np
```

```
arr = np.array([5, 15, 20])
arr1 = np.array([2, 5, 9])

print('First array:')
print(arr)

print('\nSecond array:')
print(arr1)

print('\nApplying mod() function:')
print(np.mod(arr, arr1))

print('\nApplying remainder() function:')
print(np.remainder(arr, arr1))
```

Output:

```
First array:
[ 5 15 20]

Second array:
[2 5 9]

Applying mod() function:
[1 0 2]

Applying remainder() function:
[1 0 2]
```

ARRAY FUNCTIONS

In NumPy, "array functions" refer to a wide range of operations that can be performed on NumPy arrays, including mathematical calculations, statistical analysis, manipulation of array elements, and more, allowing for efficient processing of large datasets across multiple dimensions; key functions include creating arrays like np.zeros, np.ones, arithmetic operations like addition and multiplication, statistical functions like mean, std, and indexing/slicing operations to access specific elements within an array.

Key points about NumPy array functions:

- **N-dimensional arrays:**

NumPy's core data structure is the "ndarray" (n-dimensional array), enabling operations on multidimensional data like matrices and tensors.

- **Vectorized operations:**

NumPy performs operations element-wise on entire arrays, providing significant performance advantages over traditional Python loops.

- **Universal functions (ufuncs):**

Many common mathematical functions like sin, cos, exp, log are implemented as ufuncs, allowing them to operate directly on arrays.

Example array functions in NumPy:

- **Array creation:**
    - np.array: Creates a NumPy array from a list or other iterable.
    - np.zeros: Creates an array filled with zeros
    - np.ones: Creates an array filled with ones
    - np.arange: Generates a sequence of numbers
- **Arithmetic operations:**
    - +: Addition
    - -: Subtraction
    - *: Multiplication
    - /: Division
- **Statistical functions:**
    - np.mean: Calculates the mean of array elements
    - np.std: Calculates the standard deviation

    - np.min: Finds the minimum value in an array
    - np.max: Finds the maximum value in an array
    - np.sum: Calculates the sum of array elements
- **Indexing and slicing:**
    - array[i]: Accesses the element at index i
    - array[i:j]: Extracts a slice from index i to j (exclusive)
- **Boolean indexing:**
    - array[condition] : Selects elements where a boolean condition is True
- **Linear algebra operations:**
    - np.dot: Matrix multiplication
    - np.transpose: Transposes a matrix

NumPy stands for Numerical Python. It is a Python library used for working with an array. In Python, we use the list for the array but it's slow to process. NumPy array is a powerful N-dimensional array object and is used in linear algebra, Fourier transform, and random number capabilities. It provides an array object much faster than traditional Python lists.

Types of Array:

1. One Dimensional Array
2. Multi-Dimensional Array

One Dimensional Array:

A one-dimensional array is a type of linear array.



One Dimensional Array

Example:

```
1
# importing numpy module
2
import numpy as np
3


4
# creating list
5
list = [1, 2, 3, 4]
6


7
# creating numpy array
8
sample_array = np.array(list)
9
```

```
10
print("List in python : ", list)
11


12
print("Numpy Array in python :",
13
    sample_array)
```

Output:

```
List in python :  [1, 2, 3, 4]
Numpy Array in python :  [1 2 3 4]
```

Check data type for list and array:

```
1
print(type(list_1))
2


3
print(type(sample_array))
```

Output:

```
<class 'list'>
<class 'numpy.ndarray'>
```

Multi-Dimensional Array:

Data in multidimensional arrays are stored in tabular form.

1	2	3	4	5
6	7	8	9	10

Two Dimensional Array

Example:

```
1
# importing numpy module
```

```
2
import numpy as np
3


4
# creating list
5
list_1 = [1, 2, 3, 4]
6
list_2 = [5, 6, 7, 8]
7
list_3 = [9, 10, 11, 12]
8


9
# creating numpy array
10
sample_array = np.array([list_1,
11
                         list_2,
12
                         list_3])
13


14
print("Numpy multi dimensional array in python\n",
15
      sample_array)
```

Output:

```
Numpy multi dimensional array in python
[[ 1  2  3  4]
```

[5 6 7 8]
[9 10 11 12]]

Note: use **[ ]** operators inside numpy.array() for multi-dimensional

Anatomy of an array :

1. Axis: The Axis of an array describes the order of the indexing into the array.

Axis 0 = one dimensional

Axis 1 = Two dimensional

Axis 2 = Three dimensional

2. Shape: The number of elements along with each axis. It is from a tuple.

Example:

```
1
# importing numpy module
2
import numpy as np
3


4
# creating list
5
list_1 = [1, 2, 3, 4]
6
list_2 = [5, 6, 7, 8]
7
list_3 = [9, 10, 11, 12]
8


9
# creating numpy array
10
sample_array = np.array([list_1,
11
```

```
                    list_2,
12
                    list_3])
13


14
print("Numpy array :")
15
print(sample_array)
16


17
# print shape of the array
18
print("Shape of the array :",
19
    sample_array.shape)
```

Output:

```
Numpy array :
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape of the array :  (3, 4)
```

Example:

```
1
import numpy as np
2


3
sample_array = np.array([[0, 4, 2],
4
                    [3, 4, 5],
```

```
5
                [23, 4, 5],
6
                [2, 34, 5],
7
                [5, 6, 7]])
8


9
print("shape of the array :",
10
    sample_array.shape)
```

Output:

```
shape of the array :  (5, 3)
```

3. Rank: The rank of an array is simply the number of axes (or dimensions) it has.

The one-dimensional array has rank 1.

1	2	3	4	5

Rank 1

The two-dimensional array has rank 2.

1	2	3	4	5
6	7	8	9	10

Rank 2

4. Data type objects (dtype): Data type objects (dtype) is an instance of **numpy.dtype** class. It describes how the bytes in the fixed-size block of memory corresponding to an array item should be interpreted.

Example:

```
1
# Import module
```

```
2
import numpy as np
3


4
# Creating the array
5
sample_array_1 = np.array([[0, 4, 2]])
6


7
sample_array_2 = np.array([0.2, 0.4, 2.4])
8


9
# display data type
10
print("Data type of the array 1 :",
11
    sample_array_1.dtype)
12


13
print("Data type of array 2 :",
14
    sample_array_2.dtype)
```

Output:

```
Data type of the array 1 :  int32
Data type of array 2 :  float64
```

Some different way of creating Numpy Array :

1. numpy.array(): The Numpy array object in Numpy is called ndarray. We can create ndarray using **numpy.array()** function.

Syntax: *numpy.array(parameter)*

Example:

```
1
# import module
2
import numpy as np
3


4
#creating a array
5


6
arr = np.array([3,4,5,5])
7


8
print("Array :",arr)
```

Output:

```
Array : [3 4 5 5]
```

2. numpy.fromiter(): The fromiter() function create a new one-dimensional array from an iterable object.

Syntax: *numpy.fromiter(iterable, dtype, count=-1)*

Example 1:

```
1
#Import numpy module
2
import numpy as np
3
```

```
4
# iterable
5
iterable = (a*a for a in range(8))
6

7
arr = np.fromiter(iterable, float)
8

9
print("fromiter() array :",arr)
```

Output:

fromiter() array : [0. 1. 4. 9. 16. 25. 36. 49.]

Example 2:

```
1
import numpy as np
2

3
var = "Geekforgeeks"
4

5
arr = np.fromiter(var, dtype = 'U2')
6

7
print("fromiter() array :",
```

8

arr)

Output:

fromiter() array : ['G' 'e' 'e' 'k' 'f' 'o' 'r' 'g' 'e' 'e' 'k' 's']

3. numpy.arange(): This is an inbuilt NumPy function that returns evenly spaced values within a given interval.

Syntax: *numpy.arange(start , stop, step , dtype=None)*

Example:

1

import numpy as np

2

3

np.arange(1, 20 , 2,

4

dtype = np.float32)

Output:

array([1., 3., 5., 7., 9., 11., 13., 15., 17., 19.], dtype=float32)

4. numpy.linspace(): This function returns evenly spaced numbers over a specified between two limits.

Syntax: *numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)*

Example 1:

1

import numpy as np

2

3

np.linspace(3.5, 10, 3)

Output:

```
array([ 3.5 ,  6.75, 10.  ])
```

Example 2:

```
1

import numpy as np

2


3

np.linspace(3.5, 10, 3,

4

        dtype = np.int32)
```

Output:

```
array([ 3,  6, 10])
```

5. numpy.empty(): This function create a new array of given shape and type, without initializing value.

Syntax: *numpy.empty(shape, dtype=float, order='C')*

Example:

```
1

import numpy as np

2


3

np.empty([4, 3],

4

      dtype = np.int32,

5

      order = 'f')
```

Output:

```
array([[ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11],
       [ 4,  8, 12]])
```

6. **numpy.ones():** This function is used to get a new array of given shape and type, filled with ones(1).

Syntax: *numpy.ones(shape, dtype=None, order='C')*

Example:

```
1
import numpy as np
2


3
np.ones([4, 3],
4
      dtype = np.int32,
5
      order = 'f')
```

Output:

```
array([[1, 1, 1],
     [1, 1, 1],
     [1, 1, 1],
     [1, 1, 1]])
```

7. **numpy.zeros()****:** This function is used to get a new array of given shape and type, filled with zeros(0).

Syntax: *numpy.ones(shape, dtype=None)*

Example:

```
1
import numpy as np
2
np.zeros([4, 3],
3
       dtype = np.int32,
4
       order = 'f')
```

Output:

```
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

Numpy - Array Creation

Last Updated : 24 Jan, 2025

-
-
-

Numpy Arrays are grid-like structures similar to lists in Python but optimized for numerical operations. The most straightforward way to create a NumPy array is by converting a regular Python list into an array using the np.array() function.

Let's understand this with the help of an example:

```
1

import numpy as np

2


3

# One-dimensional array

4

arr1 = np.array([1, 2, 3, 4, 5])

5

print(arr1)

6


7

# Two-dimensional array

8

arr2 = np.array([[1, 2], [3, 4]])

9
```

```
print(arr2)
```

Output

```
[1 2 3 4 5]

[[1 2]

 [3 4]]
```

Creating Arrays with Specific Values

For assigning a specific values. NumPy provides several function to create arrays filled with zeros, ones, or a specific constant value.

- **Zeros Array:** np.zeros() function creates an array filled with zeros. It requires the shape of the array as a parameter.

Example:

```
1

import numpy as np

2


3

# 3x4 array filled with zeros

4

arr_zero = np.zeros((3, 4))

5

print(arr_zero)
```

Output

```
[[0. 0. 0. 0.]

 [0. 0. 0. 0.]

 [0. 0. 0. 0.]]
```

- **Ones Array:** np.ones() creates an array filled with ones.

Example:

```
1

import numpy as np

2
```

3

2x3 array filled with ones

4

arr_one = np.ones((2, 3))

5

print(arr_one)

Output

[[1. 1. 1.]

 [1. 1. 1.]]

- **Full Array :** np.full() function allows you to create an array filled with a specific value.

Example:

1

import numpy as np

2

3

2x2 array filled with 7

4

arr_full = np.full((2, 2), 7)

5

print(arr_full)

Output

[[7 7]

 [7 7]]

Creating Arrays with Random Values

NumPy also has functions for generating arrays with random values, useful for simulations and testing.

- **Random Float Array :** np.random.rand() function generates an array of random values between 0 and 1.

Example:

```
1

import numpy as np

2


3

 # 2x3 array of random floats

4

arr_rand = np.random.rand(2, 3)

5

print(arr_rand)
```

Output

```
[[0.67820861 0.64484802 0.48673431]

 [0.00263043 0.55383721 0.43240166]]
```

- **Random Integers :** If we need random integers, we can use **np.random.randint()** to create arrays with integer values in a specified range.

Example:

```
1

import numpy as np

2


3

 # 3x3 array of random integers from 1 to 9

4

arr_int = np.random.randint(1, 10, size=(3, 3))

5

print(arr_int)
```

Output

[[4 6 5]

[7 4 8]

[8 5 2]]

Creating Arrays with a Range of Values

Another common method of creating arrays is using a range of values. NumPy provides functions like np.arange() and np.linspace() for this purpose.

- **Using np.arange() :** np.arange() creates arrays with values spaced according to a given interval. It's similar to Python's built-in range() but returns a NumPy array.

Example:

1

import numpy as np

2

3

Array from 0 to 10 with step 2

4

arr_range = np.arange(0, 10, 2)

5

print(arr_range)

Output

[0 2 4 6 8]

- **Using np.linspace():** np.linspace() generates an array with a specified number of evenly spaced values over a specified range.

Example:

1

import numpy as np

2

```
3
# 5 values from 0 to 1
4
arr_linspace = np.linspace(0, 1, 5)
5
print(arr_linspace)
```

Output

```
[0.   0.25 0.5  0.75 1.  ]
```

Identity and Diagonal Matrices

NumPy also provides functions for creating identity matrices and diagonal matrices, which are often used in linear algebra.

- **Identity Matrix :** np.eye() function creates an identity matrix, a square matrix with ones on the diagonal and zeros elsewhere.

Example:

```
1
import numpy as np
2


3
# 3x3 identity matrix
4
identity_matrix = np.eye(3)
5
print(identity_matrix)
```

Output

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

- **Diagonal Matrix :** Use np.diag() to create a diagonal matrix, where the provided array elements form the diagonal.

Example:

```
1
import numpy as np
2


3
# Diagonal matrix with [1, 2, 3] on the diagonal
4
diag_matrix = np.diag([1, 2, 3])
5
print(diag_matrix)
```

Output

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

Methods for array creation in Numpy

Function	Description
empty()	Return a new array of given shape and type, without initializing entries
empty_like()	Return a new array with the same shape and type as a given array
eye()	Return a 2-D array with ones on the diagonal and zeros elsewhere.
identity()	Return the identity array

Function	Description
ones()	Return a new array of given shape and type, filled with ones
ones_like()	Return an array of ones with the same shape and type as a given array
zeros()	Return a new array of given shape and type, filled with zeros
zeros_like()	Return an array of zeros with the same shape and type as a given array
full_like()	Return a full array with the same shape and type as a given array.
array()	Create an array
asarray()	Convert the input to an array
asanyarray()	Convert the input to an ndarray, but pass ndarray subclasses through
ascontiguousarray()	Return a contiguous array in memory (C order)
asmatrix()	Interpret the input as a matrix
copy()	Return an array copy of the given object
frombuffer()	Interpret a buffer as a 1-dimensional array

Function	Description
fromfile()	Construct an array from data in a text or binary file
fromfunction()	Construct an array by executing a function over each coordinate
fromiter()	Create a new 1-dimensional array from an iterable object
fromstring()	A new 1-D array initialized from text data in a string
loadtxt()	Load data from a text file
arange()	Return evenly spaced values within a given interval
linspace()	Return evenly spaced numbers over a specified interval
logspace()	Return numbers spaced evenly on a log scale
geomspace()	Return numbers spaced evenly on a log scale (a geometric progression)
meshgrid()	Return coordinate matrices from coordinate vectors
mgrid()	nd_grid instance which returns a dense multi-dimensional “meshgrid
ogrid()	nd_grid instance which returns an open multi-dimensional “meshgrid
diag()	Extract a diagonal or construct a diagonal array

Function	Description
diagflat()	Create a two-dimensional array with the flattened input as a diagonal
tri()	An array with ones at and below the given diagonal and zeros elsewhere
tril()	Lower triangle of an array
triu()	Upper triangle of an array
vander()	Generate a Vandermonde matrix
mat()	Interpret the input as a matrix
bmat()	Build a matrix object from a string, nested sequence, or array

Saving and loading NumPy Arrays

Last Updated : 26 Dec, 2023

-
-
-

The savetxt() and loadtxt() functions in NumPy are primarily designed for 1D and 2D arrays (text files with row/column format). When dealing with a 3D NumPy array, these functions can be a bit limited because they cannot directly handle the 3D structure. However, you can reshape the 3D array into a 2D array, save it, and then reshape it back to its original form upon loading. In this article, we will see how to load and save 3D NumPy Array to file using savetxt() and loadtxt() functions and NumPy loadtxt and savetxt usage guide.

Load and Save 3D Numpy Array to File

Below are the ways by which we can load and save 3D NumPy array to file using savetxt() and loadtxt() functions in Python:

- Utilize the savetxt() and loadtxt() functions for TXT files
- Saving and loading the 3D arrays(reshaped) into CSV files

Example 1: Saving a 3D Numpy Array as a Text File

In this example, a 3D NumPy array arr is reshaped into a 2D format and saved to a text file named “geekfile.txt” using savetxt(). Later, the data is retrieved from the file, reshaped back to its original 3D form, and compared with the original array to verify its equality.

- Python3

```
import numpy as gfg


arr = gfg.random.rand(5, 4, 3)


# reshaping the array from 3D

# matrice to 2D matrice.

arr_reshaped = arr.reshape(arr.shape[0], -1)
```

```
# saving reshaped array to file.
gfg.savetxt("geekfile.txt", arr_reshaped)

# retrieving data from file.
loaded_arr = gfg.loadtxt("geekfile.txt")

load_original_arr = loaded_arr.reshape(
    loaded_arr.shape[0], loaded_arr.shape[1] // arr.shape[2], arr.shape[2])

# check the shapes:
print("shape of arr: ", arr.shape)
print("shape of load_original_arr: ", load_original_arr.shape)

# check if both arrays are same or not:
if (load_original_arr == arr).all():
    print("Yes, both the arrays are same")
else:
    print("No, both the arrays are not same")
```

Output:

shape of arr: (5, 4, 3)

shape of load_original_arr: (5, 4, 3)

Yes, both the arrays are same

Example 2: Saving and loading the 3D arrays(reshaped) into CSV files

In this example, we will perform saving and loading the 3D arrays(reshaped) into CSV files by using savetxt and loadtxt functions respectively. Here, a random 3D NumPy array arr is reshaped into a 2D format, saved as a CSV file, and then loaded back into a 2D array. The loaded data is reshaped back to its original 3D form, and a comparison is made with the original array to confirm their equality.

- Python3

```
import numpy as np
```

```
# Create a sample 3D array
arr = np.random.rand(5, 4, 3)

# Reshape the 3D array to 2D
arr_reshaped = arr.reshape(arr.shape[0], -1)

# Save the 2D array to a CSV file
np.savetxt("3d_array.csv", arr_reshaped, delimiter=",")

# Load the 2D array from the CSV file
loaded_arr = np.loadtxt("3d_array.csv", delimiter=",")

# Reshape the 2D array back to its original 3D shape
load_original_arr = loaded_arr.reshape((arr.shape[0], arr.shape[1], arr.shape[2]))

# Verify if the loaded array matches the original
if np.array_equal(load_original_arr, arr):
    print("Yes, both the arrays are the same")
else:
    print("No, both the arrays are not the same")
```

Output:

Yes, both the arrays are same

NumPy save() Method | Save Array to a File

Last Updated : 02 Feb, 2024

-
-
-

The NumPy **save()** method is used to **store the input array** in a binary file with the '**npy extension'** (.npy).

Example:

- Python3

```
import numpy as np

a = np.arange(5)

np.save('array_file', a)
```

Syntax

Syntax: *numpy.save(file, arr, allow_pickle=True, fix_imports=True)*

Parameters:

- ***file:*** *File or filename to which the data is saved. If the file is a string or Path, a .npy extension will be appended to the file name if it does not already have one. If the file is a file object, then the filename is unchanged.*
- ***allow_pickle :*** *Allow saving object arrays using Python pickles. Reasons for disallowing pickles include security (loading pickled data can execute arbitrary code) and portability (pickled objects may not be loadable on different Python installations). Default: True*
- ***fix_imports :*** *Only useful in forcing objects in object arrays on Python 3 to be pickled in a Python 2 compatible way.*
- ***arr :*** *Array data to be saved.*

Returns: *Stores the input array in a disk file with '.npy' extension.*

Examples

Let's understand the workings of numpy.save() method in these Python code and know how to use save() method of NumPy library.

To use numpy.save() function, you just need to pass the file name and array in the function.

Example 1

- Python3

```
# Python program explaining
# save() function

import numpy as geek

a = geek.arange(5)

# a is printed.
print("a is:")
print(a)

# the array is saved in the file geekfile.npy
geek.save('geekfile', a)

print("the array is saved in the file geekfile.npy")
```

Output :

a is:

[0 1 2 3 4]

the array is saved in the file geekfile.npy

Example 2

- Python3

```
# Python program explaining
# save() function

import numpy as geek

# the array is loaded into b
b = geek.load('geekfile.npy')
```

```
print("b is:")
print(b)

# b is printed from geekfile.npy
print("b is printed from geekfile.npy")
```

Output :

b is:

[0 1 2 3 4]

b is printed from geekfile.npy

NUMPY RANDOM NUMBER

What is a Random Number?

Random number does NOT mean a different number every time. Random means something that can not be predicted logically.

Pseudo Random and True Random.

Computers work on programs, and programs are definitive set of instructions. So it means there must be some algorithm to generate a random number as well.

If there is a program to generate random number it can be predicted, thus it is not truly random.

Random numbers generated through a generation algorithm are called *pseudo random*.

Can we make truly random numbers?

Yes. In order to generate a truly random number on our computers we need to get the random data from some outside source. This outside source is generally our keystrokes, mouse movements, data on network etc.

We do not need truly random numbers, unless it is related to security (e.g. encryption keys) or the basis of application is the randomness (e.g. Digital roulette wheels).

Generate Random Number

NumPy offers the random module to work with random numbers.

ExampleGet your own Python Server

Generate a random integer from 0 to 100:

```
from numpy import random

x = random.randint(100)

print(x)
```

Generate Random Float

The random module's rand() method returns a random float between 0 and 1.

Example

Generate a random float from 0 to 1:

```
from numpy import random

x = random.rand()

print(x)
```

Generate Random Array

In NumPy we work with arrays, and you can use the two methods from the above examples to make random arrays.

Integers

The randint() method takes a size parameter where you can specify the shape of an array.

Example

Generate a 1-D array containing 5 random integers from 0 to 100:

```
from numpy import random

x=random.randint(100, size=(5))

print(x)
```

Example

Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

```
from numpy import random

x = random.randint(100, size=(3, 5))

print(x)
```

Floats

The rand() method also allows you to specify the shape of the array.

Example

Generate a 1-D array containing 5 random floats:

```
from numpy import random

x = random.rand(5)

print(x)
```

Example

Generate a 2-D array with 3 rows, each row containing 5 random numbers:

```
from numpy import random

x = random.rand(3, 5)

print(x)
```

Generate Random Number From Array

The choice() method allows you to generate a random value based on an array of values.

The choice() method takes an array as a parameter and randomly returns one of the values.

Example

Return one of the values in an array:

```
from numpy import random

x = random.choice([3, 5, 7, 9])

print(x)
```

The choice() method also allows you to return an *array* of values.

Add a size parameter to specify the shape of the array.

Example

Generate a 2-D array that consists of the values in the array parameter (3, 5, 7, and 9):

```
from numpy import random
```

```
x = random.choice([3, 5, 7, 9], size=(3, 5))

print(x)
```

Numpy Random

In NumPy, we have a module called random which provides functions for generating random numbers.

These functions can be useful for generating random inputs for testing algorithms.

Generate Random Integer in NumPy

As discussed earlier, we use the random module to work with random numbers in NumPy.

Let's see an example.

```
import numpy as np


# generate random integer from 0 to 9

random_number = np.random.randint(0, 10)


print(random_number)


# Output: 7
```

In this example, we have used the random module to generate a random number. The random.randint() function takes two arguments,

- **0** - a lower bound (inclusive)
- **10** - an upper bound (exclusive)

Here, random.randint() returns a random integer between **0** and **9**.

Since the output will be a randomly generated integer between **0** and **9**, we will see different outputs each time the code is run.

Note: We can also import and use the random module like this:

```
from numpy import random


random_number = random.randint(0, 10)


print(random_number)
```

Here, the syntax is slightly different but the output will be the same as above, we will get a random integer between **0** and **9**.

Generate Random Float in NumPy

We can also generate a random floating-point number between **0** and **1**. For that we use the random.rand() function. For example,

```
import numpy as np


# generate random float-point number between 0 and 1

random_number = np.random.rand()


print(random_number)


# Output: 0.7696638323107154
```

Here, random.rand() generates a random floating-point number between **0** and **1**.

Since the number is generated randomly, the output value can vary each time the code is run.

Generate Random Array in NumPy

NumPy's random module can also be used to generate an array of random numbers. For example,

```
import numpy as np
```

```
# generate 1D array of 5 random integers between 0 and 9
integer_array = np.random.randint(0, 10, 5)

print("1D Random Integer Array:\n",integer_array)

# generate 1D array of 5 random numbers between 0 and 1
float_array = np.random.rand(5)

print("\n1D Random Float Array:\n",float_array)

# generate 2D array of shape (3, 4) with random integers
result = np.random.randint(0, 10, (3,4))

print("\n2D Random Integer Array:\n",result)
```

Output

```
1D Random Integer Array:
[9 7 8 4 2]

1D Random Float Array:
 [0.7877579  0.01723754 0.93995075 0.17126388 0.69913594]

2D Random Integer Array:
 [[0 5 3 8]
 [3 9 2 1]
 [8 7 1 2]]
```

Here,

- np.random.randint(0, 10, 5) - generates a 1D array of **5** random integers between **0** and **9**.

- np.random.rand(5) - generates a 1D array of **5** random numbers between **0** and **1**.
- np.random.randint(0, 10, (3,4)) - generates a 2D array of shape **(3, 4)** with random integers between **0** and **9**.

Choose Random Number from NumPy Array

To choose a random number from a NumPy array, we can use the random.choice() function.

Let's see an example.

```
import numpy as np


# create an array of integers from 1 to 5

array1 = np.array([1, 2, 3, 4, 5])


# choose a random number from array1

random_choice = np.random.choice(array1)


print(random_choice)


# Output: 3
```

In the above example, the np.random.choice(array1) function chooses a random number from the array1 array.

It is important to note that the output will be a single random number from array1, which will be different each time the code is run.